

HELEP — L3 Patterns-in-Code Document

Software Architecture — Orchestration with Kubernetes

Part A — Pre-implemented Patterns

The following six patterns are already present in the HELEP starter code. Each entry includes a code citation (file:line) and an explanation of how the pattern is applied in context.

A.1 Choreographed Saga

Where:

sos-service/app/main.py (lines 78-100), dispatch-service/app/main.py (lines 54-90), notification-service/app/main.py

How it works:

The saga spans three services without a central coordinator. When a citizen triggers an SOS, sos-service inserts an incident row into SQLite and immediately publishes sos.triggered to Kafka (sos-service/app/main.py:88-98). dispatch-service consumes that event, picks the nearest free responder, and emits responder.assigned (dispatch-service/app/main.py:68-80). notification-service then consumes responder.assigned and logs an SMS notification. Each service reacts to events — no service calls another directly.

Compensation step:

When the victim cancels (POST /sos/{id}/cancel), sos-service publishes sos.cancelled (sos-service/app/main.py:104-110). dispatch-service consumes this event, calls release_assignment() to set the responder busy=0 and marks the assignment row RELEASED (dispatch-service/app/main.py:83-90). The rollback trigger is the sos.cancelled Kafka event, keyed by incident_id to preserve ordering on the same partition.

A.2 Pub/Sub via Apache Kafka

Where:

app/events.py in every service — AIOKafkaProducer (line 28) + AIOKafkaConsumer (line 75)

Consumer group semantics (at-least-once):

Auto-commit is disabled (enable_auto_commit=False, events.py:81). The consumer only calls await consumer.commit() AFTER the handler succeeds (events.py:91). If the handler throws, the offset is not committed, so the message is re-delivered on the next poll. This guarantees at-least-once delivery — no event is silently dropped.

Partition keying:

Every saga-critical publish passes key=incident_id (e.g. sos-service/app/main.py:90, dispatch-service/app/main.py:71). Kafka routes all events with the same key to the same partition. Since one partition is owned by exactly one consumer in a group, all events for one SOS are processed in order, preventing double-dispatch even with multiple replicas.

A.3 Repository

Where:

app/db.py in every service — e.g. dispatch-service/app/db.py (lines 40-110), user-service/app/db.py (lines 30-80)

Why:

The Repository pattern isolates all SQLite access behind named functions (insert_user, find_by_phone, reserve_responder_for, etc.). Route handlers never write SQL directly — they call db functions. This separation means the storage layer can be swapped (e.g. SQLite to PostgreSQL) without touching business logic. It also makes the atomic UPDATE ... WHERE busy=0 constraint (dispatch-service/app/db.py:75) reusable and testable in isolation. Without this pattern, SQL scattered across handlers would make the no-double-dispatch invariant impossible to enforce consistently.

A.4 Strategy

Where:

dispatch-service/app/matching.py — Matcher Protocol (line 19), NearestMatcher (line 23), CredibilityWeightedMatcher (line 32)

How to switch:

Set the MATCHER environment variable: nearest (default) or credibility. The factory function matcher() (line 52) reads MATCHER and returns the appropriate implementation. In K8s this is set in the Deployment env spec.

Added third strategy — RoundRobinMatcher:

```
# dispatch-service/app/matching.py (added lines 44-51)
class RoundRobinMatcher:
    _index = 0
    def pick(self, vlat, vlon, responders):
        pool = list(responders)
        if not pool: return None
        r = pool[RoundRobinMatcher._index % len(pool)]
        RoundRobinMatcher._index += 1
        return {'id': r['id'], 'distance_m': 0}
```

Factory updated: elif name == 'roundrobin': return RoundRobinMatcher() (matching.py:56). This enables fair load distribution independent of geography — useful when all responders are equidistant.

A.5 Outbox-lite

Where:

sos-service/app/main.py — trigger() function (lines 78-100)

How it works:

```
# sos-service/app/main.py:85-98
insert_incident(iid, claims['sub'], body.lat, body.lon, ...) # db write first
await publish('sos.triggered', {...}, key=iid) # then publish
```

The DB insert and Kafka publish happen in the same async block. If publish fails, the DB row still exists and can be replayed. This is 'lite' because a real Outbox would write the event to a dedicated outbox table in the same DB transaction as the business record, then a separate relay process would publish to Kafka and mark it sent — eliminating the dual-write gap entirely. The lite version still has a small window where the DB write succeeds but Kafka publish fails before the row can be replayed.

A.6 Circuit Breaker (stub — completed)

Where:

every service's app/events.py — class CircuitBreaker (lines 55-72)

Completed state machine:

```
import time
class CircuitBreaker:
    def __init__(self, fail_threshold=5, reset_after_s=10.0):
        self.fail_threshold = fail_threshold
        self.reset_after_s = reset_after_s
        self.fails = 0
```

```

        self.opened_at = None

    def allow(self) -> bool:
        if self.opened_at is None:          # CLOSED
            return True
        elapsed = time.time() - self.opened_at
        if elapsed >= self.reset_after_s:    # HALF-OPEN
            return True
        return False                        # OPEN

    def record_success(self):
        self.fails = 0
        self.opened_at = None               # back to CLOSED

    def record_failure(self):
        self.fails += 1
        if self.fails >= self.fail_threshold:
            self.opened_at = time.time()     # transition to OPEN

```

State transitions: CLOSED (opened_at=None, allows all) → OPEN (fail_threshold reached, blocks all) → HALF-OPEN (reset_after_s elapsed, allows one probe). If the probe succeeds, record_success() resets to CLOSED; if it fails, record_failure() re-opens. This prevents cascading failures when Kafka becomes temporarily unreachable.

Part B — Patterns Added (minimum 2)

B.1 CQRS — Command Query Responsibility Segregation

Where added:

analytics-service/app/main.py and analytics-service/app/db.py

Problem it solves in HELEP:

All five services write events to their own SQLite databases (command side). analytics-service acts as a dedicated read model — it consumes all Kafka topics and maintains its own counters table (analytics-service/app/db.py). Police and admin query only analytics-service (GET /stats/events, GET /stats/zones) without touching the operational databases. This separates read load from write load and allows the analytics read model to be structured for querying (aggregated counters, zone heatmaps) rather than for transactional integrity.

Trade-off vs. shared database:

A shared database would give consistent reads immediately, but violates service isolation and creates a single point of failure. CQRS with Kafka introduces eventual consistency (analytics may lag by one Kafka poll interval), which is acceptable for statistical views but not for real-time dispatch decisions.

B.2 Bulkhead — Separate Consumer Groups per Service

Where added:

Each service's app/main.py consumer call — GROUP = 'dispatch-service' (dispatch-service/app/main.py:18), GROUP = 'notification-service', GROUP = 'analytics-service'

Problem it solves in HELEP:

Each service registers its own Kafka consumer group. This means a failure or slowdown in analytics-service (e.g. heavy aggregation query) does not affect dispatch-service's consumption rate. Each group maintains its own offset per topic partition, so one slow consumer never blocks another. This is the Bulkhead pattern: failure in one partition of the system is contained and cannot cascade to others.

Trade-off vs. shared consumer group:

A shared group would mean only one service processes each message — breaking the fan-out requirement where `sos.triggered` must reach both dispatch AND notification AND analytics. Separate groups duplicate message delivery but guarantee isolation and independent scaling.

Part C — Anti-patterns Avoided

Shared Database Anti-pattern — Avoided

HELEP explicitly avoids the shared database anti-pattern. Each service owns its own SQLite file at a unique `DB_PATH` (`user.db`, `sos.db`, `dispatch.db`, `notification.db`, `analytics.db`), declared in each service's `db.py`:
`DB_PATH = os.getenv('DB_PATH', '/data/X.db')` (e.g. `dispatch-service/app/db.py:line 8`, `user-service/app/db.py:line 8`).

A shared database would create tight coupling between services — a schema change in one service could break others, and a DB outage would cascade across the entire system. By giving each service its own database, HELEP enforces bounded contexts: `dispatch-service` owns responders and assignments; `user-service` owns identity. Services communicate only via Kafka events, never via direct DB access. In the K8s deployment, each service gets its own `PersistentVolumeClaim`, making the isolation explicit at the infrastructure level.

Code excerpts are kept under 10 lines as required. All citations reference the HELEP starter code at github.com/MATRIX30/Helep.